

Exercise 2 — Codebase Exploration Challenge

Exercise Description

Apply three prompt strategies to understand existing code **without making changes**. Work with the Task Manager (Python) and keep a "Code Understanding Journal" covering: how a specific feature works, deepening understanding through guided questions, and mapping data flow. Finish with a short presentation of the architecture.

Codebase used: `ai-code-exercises/use-cases/code-algorithms/python/TaskManager`.

Code Understanding Journal

Part 1 — How task creation & status updates work

Files involved: `cli.py` → `task_manager.py` → `storage.py` → `models.py`.

Creation flow (`create`): 1. `cli.main()` parses args, splits comma-separated tags, and calls `TaskManager.create_task(title, description, priority, due, tags)`. 2. `create_task` converts the integer priority to `TaskPriority(value)`, parses the due date (`strptime %Y-%m-%d`, returns `None` and prints an error on bad input), constructs a `Task`, and calls `storage.add_task(task)`. 3. `Task.__init__` assigns a UUID, sets `status = TODO`, and stamps `created_at == updated_at`. 4. `TaskStorage.add_task` stores the task in the in-memory dict and immediately calls `save()` (writes the whole dict to `tasks.json` via `TaskEncoder`). Returns the id.

Status-update flow (`status`): - `TaskManager.update_task_status` special-cases `DONE`: it fetches the task, calls `task.mark_as_done()` (sets `DONE`, stamps `completed_at / updated_at`) and saves. For any other status it delegates to `storage.update_task(task_id, status=...)`, which calls `Task.update(**kwargs)` and saves.

Design patterns observed: clear **layered architecture**; **Repository-like** `TaskStorage`; **enum-driven** state. **Storage-per-write** (every mutation persists the whole file) — simple but not scalable.

Journal note / edge case: `update_task_status` returns `None` (falsy) if a `DONE` update targets a missing task, but the non-`DONE` branch returns `storage.update_task`'s boolean. Slight inconsistency worth a test.

Part 2 — Deepening understanding via guided questions (task prioritisation)

Questions I worked through against `task_priority.py`:

- *What is the score composed of?* Base = priority weight (LOW 1, MEDIUM 2, HIGH 4, URGENT 6) × 10, then additive due-date urgency (overdue +35 down to +10 for "within a week"), status penalties (DONE −50, REVIEW −15), a +8 tag boost for blocker/critical/urgent, and +5 if updated in the last day.
- *Is the priority weighting linear?* No — note the jump HIGH(4)→URGENT(6) is bigger than LOW→MEDIUM→HIGH steps, so URGENT is deliberately emphasised.
- *What happens for a DONE urgent task?* $6 \times 10 + (-50) = 10$, minus any due bonus interplay — DONE tasks are pushed down hard, as intended.
- *Sorting:* `sort_tasks_by_importance` builds `(score, task)` tuples and sorts by score descending using `key=lambda x: x[0]` (correctly avoids comparing `Task` objects when scores tie).

Misconception corrected: I first assumed priority alone dominated; in fact an overdue MEDIUM task ($20 + 35 = 55$) outranks a non-urgent HIGH task with no due date (40), so **due-date urgency can outweigh priority** — an important product behaviour.

Part 3 — Mapping data flow for "mark a task complete"

```
CLI: status <id> done
↳ TaskManager.update_task_status(id, "done")
    ↳ TaskStatus("done") == DONE → special path
        ↳ task = storage.get_task(id)
            ↳ task.mark_as_done()      # status=DONE, completed_at=now, updated_at=now
                ↳ storage.save()      # serialise ALL tasks → tasks.json
returns True
```

- **State changes:** `status` → DONE, `completed_at` set (was None), `updated_at` bumped.
- **Points of failure:** unknown id (special path returns None, no explicit "not found" message from the manager); a write error in `save()` is caught and printed but the in-memory state has already changed, so memory and disk can diverge; concurrent runs would clobber `tasks.json` (last-writer-wins).
- **Persistence:** whole-file rewrite via `TaskEncoder` on every change.

Part 4 — Architecture Presentation (speaker notes, ~3-5 min)

1. **High level:** a dependency-free Python CLI task manager with a clean layered design: CLI → `TaskManager` (service) → `TaskStorage` (repository) → `Task/enums` (domain). Three standalone algorithm modules (scoring, parsing, merge) operate on domain objects.

2. **Three features:** *Creation* builds a domain object and persists it; *Prioritisation* is a pure, additive scoring function combining priority, due-date urgency, status and tags; *Completion* is a state transition that stamps `completed_at` and rewrites the store.
 3. **Interesting pattern:** the **Repository** abstraction (`TaskStorage`) fully hides JSON/`datetime`/enum serialisation from the service layer — the manager never sees JSON.
 4. **Hardest part & how prompts helped:** the scoring interplay between priority and due-date was non-obvious; asking the AI to trace concrete tasks (overdue MEDIUM vs. HIGH-no-due) revealed the real ranking behaviour.
-

Submission for Exercise 2 — Codebase Exploration Challenge

1. **Part 1 — task creation/status updates:** components (CLI → `TaskManager` → `TaskStorage` → `Task`), the create and DONE/non-DONE update flows, storage-per-write persistence, and the Repository/enum-state patterns (plus a noted return-value inconsistency).
2. **Part 2 — prioritisation:** initial assumption that priority dominates was wrong; discovered due-date urgency can outweigh priority, and the deliberately non-linear URGENT weighting; confirmed the tuple-sort avoids comparing `Task` objects.
3. **Part 3 — completion data flow:** full call chain with the three state changes (status, `completed_at`, `updated_at`), failure points (missing id, memory/disk divergence on write error, concurrent last-writer-wins), and whole-file persistence.
4. **Part 4 — presentation:** architecture overview, the three feature explanations, the Repository pattern as the standout design choice, and the scoring interplay as the hardest thing to grasp (resolved by tracing concrete examples with AI).